# Mumbai Pav Co. — Production Engineering & Technical Architecture Specification

**Project Name:** Mumbai Pav Co. (Authentic Street-Food Enterprise Platform)
**Flagship Hub:** Koramangala 5th Block, Bengaluru, Karnataka, India
**System Version:** `1.0.0-PROD-STABLE`
**Document Author:** Principal Full-Stack Cloud Architect & Lead Systems Engineer
**Target Audience:** Engineering Leads, DevOps Engineers, Product Managers, Store Operations Managers

## 1. Executive Summary & Architectural Vision

**Mumbai Pav Co.** is an authentic street-food venture platform built to solve the high-volume operational challenges of fast-casual Indian dining. Unlike standard e-commerce or generic restaurant templates, street food operations (especially artisanal Batata Vada Pav and Masala Chai) require:

1. **Ultra-Fast Batch Frying Cycles:** Fresh vadas are fried in small batches every 8–15 minutes. The software must pace order intake, display real-time prep queues, and warn kitchen chefs before tickets exceed the 8-minute freshness SLA.
2. **Bill of Materials (BOM) Raw Inventory Depletion:** Selling a single Vada Pav is not just an item deduction; it triggers automated fractional depletion across raw agricultural ingredients (potatoes, gram flour/besan, fresh pav loaves, butter, and chillies).
3. **Role-Segregated Multi-Station Flow:** Public customers, fry chefs at the kadai, counter cashiers handling walk-in cash/UPI, and store owners managing financial ledgers operate on dedicated interfaces with zero route cross-contamination.
4. **Frictionless Omnichannel Ordering:** Seamless support for **Takeaway counter pickups**, **Dine-In Table QR scans**, and **Direct Delivery** geofenced to Bengaluru postal codes.

## 2. Complete Technology Stack & Architectural Decisions

### Architectural Decision Records (ADRs)

| Technology | Selection | Architectural Rationale |
|---|---|---|
| **Framework** | **Next.js 14 (App Router)** | Provides unified full-stack architecture: React Server Components for initial rendering, API Route Handlers with low execution overhead, and Next.js Edge Middleware for zero-latency role guards. |
| **Language** | **TypeScript 5 (Strict Mode)** | End-to-end type safety eliminating runtime shape mismatches between API response bodies, database entities, and client form states. |
| **Styling** | **Tailwind CSS v3** | Utility-first architecture allowing precise breakpoint control ( `sm:` , `md:` , `lg:` , `xl:` ) without writing bloated custom CSS stylesheets. Integrated dark theme palette ( `slate-950` ). |
| **Database & ORM** | **Prisma ORM 5.22.0** | Type-safe declarative modeling, relational cascades, automated migration history, and cross-database portability (SQLite for development, PostgreSQL for enterprise cloud). |
| **Validation** | **Zod v3** | Strict runtime schema parsing on incoming payloads ( `/api/orders` , `/api/payment/verify` ), preventing SQL injection, parameter tampering, and type coercion bugs. |
| **Audio Alerting** | **Web Audio API Synthesizer** | Pure browser audio synthesis using `OscillatorNode` and `GainNode` . Produces warm harmonic bell chimes and tactile click ticks without downloading external MP3/WAV assets. |
| **Visual Delight** | **Canvas Confetti** | Hardware-accelerated canvas particle bursts triggering immediately upon ticket completion ( `READY_FOR_PICKUP` ). |

## 3. Database Architecture & Relational Schema (Prisma)

The database schema is normalized and indexed for high-volume transactions:

### Core Schema Models Detailed

1. `User` : Supports phone-based customer authentication, loyalty points ledger (e.g. 50 bonus points on registration), and role flags ( `CUSTOMER` , `KITCHEN_STAFF` , `CASHIER` , `ADMIN` ).
2. `MenuItem` & `Category` : Structured catalog supporting high-res imagery, preparation duration benchmarks, vegetarian flags, spice level categorization ( `MILD` , `MEDIUM` , `THECHA_HOT` ), and bestseller flags.

3. `Modifier`: Add-ons such as Melted Amul Cheese Slice (+₹20), Extra Butter Grilled Pav (+₹15), and Extra Dry Garlic Thecha Chutney (+₹10).
4. `InventoryItem` & `RecipeBOM` **(Bill of Materials)**:
    - Tracks raw culinary staples: Batata (Potatoes), Pav Loaves, Besan, Fresh Green Chillies, Amul Butter, Groundnut Frying Oil, Tea Leaves, and Whole Milk.
    - Every completed order triggers an atomic relational decrement according to its BOM recipe ratios.
5. `Order` & `OrderItem`: Captures unique alphanumeric order numbers (e.g. `MPC-20261022-381`), high-visibility sequential counter tokens (`#101`, `#102`), delivery metadata, table numbers, and full item snapshots.
6. `Transaction` & `AuditLog`: Stores Razorpay payment signatures, payment gateway references, gateway responses, and audit records for staff transitions.
7. `StoreSetting`: Global store configuration singleton controlling store open/close master state, surge order throttle pacing, minimum order limits, and marquee notices.

## 4. Multi-Role Station Architecture & Operational Flows

### Station 1: Customer Storefront (`/`)

- **Responsive Hero & Menu**: Dynamic category navigation pills (`All`, `Signature Vada Pav`, `Street Combos`, `Chai & Coolers`, `Sides & Thecha`).
- **Interactive BOM Cost Inspector**: Customers and store managers can inspect the raw ingredient cost profile and gross margin of any item right on the catalog card.
- **Customization Modal / Native Bottom Sheet**:
    - Modal switches to a native bottom sheet on mobile devices (`items-end`, `max-h-[88vh]`).
    - Allows selecting spice level (`Mild / Meetha`, `Mumbai Classic`, `Thecha Hot` 🔥) and extra add-ons with real-time price updates.
- **Cart Drawer**:
    - Full-width slide-out drawer on mobile screens; 448px panel on desktop.
    - Dining mode selection: **Takeaway** (with pickup slot scheduler), **Dine-In** (table QR auto-fill), and **Direct Delivery** (Bengaluru PIN code validator).
    - Built-in Coupon Code validation engine (`PAV50`, `BUMP100`, `FREECHAI`).

- Auto-calculated 5% Composite GST on food services.

### Station 2: Live Digital Token Tracker ( `/order/[id]` )

- **Digital Counter Token Banner**: Sized dynamically ( `text-5xl sm:text-7xl` ) to ensure high visibility without overflow on narrow screens.
- **Status Lifecycle Tracker**:
  1. `Order Confirmed` ➔ Ticket received at Koramangala hub.
  2. `Kadai Frying` ➔ Fresh batata vadas and butter pav in the fryer.
  3. `Ready at Counter / Packed for Rider` ➔ Digital buzzer triggers.
  4. `Order Completed` ➔ Delivered or picked up.
- **Audio Synthesizer & Confetti**: Plays an acoustic chime and fires celebratory confetti particles when the order enters the `READY_FOR_PICKUP` status.
- **Real-Time Polling**: 3-second background polling cycle syncing with the kitchen database.

### Station 3: Kitchen Display System ( `/kds` )

- **Kanban Board**: Three color-coded workflow columns:
  - `Incoming New Orders` (Amber)
  - `In Preparation / Kadai Frying` (Orange)
  - `Ready for Pickup / Packed` (Emerald)
- **8-Minute SLA Warning System**: Orders in the queue for 5+ minutes show an amber badge; orders exceeding 8 minutes pulse with a red warning border and audio alert to prevent delayed tickets.
- **One-Tap Actions**:
  - `Start Frying` (Transitions state to `PREPARING` ).
  - `Mark Ready` (Transitions state to `READY_FOR_PICKUP` ).
  - `Print KOT` (Thermal 80mm kitchen order ticket layout).
- **Mobile Responsive Tabs**: On phones and small tablets, kitchen staff can toggle columns ( `All` , `New` , `Frying` , `Ready` ) to avoid squished columns.

### Station 4: Express Counter POS Register ( `/pos` )

- **Rapid Touch Grid**: Cashiers can punch items with a single tap.
- **Payment Modalities**: One-click toggle between `COUNTER_CASH` and `COUNTER_UPI` with dynamic QR preview.

- **Auto-Printing Thermal Receipt**: Formatted for standard 80mm ESC/POS thermal receipt printers with store GSTIN, itemized taxes, and token number.
- **Mobile Jump Action**: Cashiers on mobile devices get a floating jump button to instantly scroll down to the current register bill.

### Station 5: Store Owner & Operations HQ ( `/admin` )

- **Operational Master Toggles**:
  - `Store Open / Closed`: Instantly disables online order intake across all client storefronts.
  - `Peak Surge Throttle`: Dynamically throttles order intake during rush hours to prevent kitchen bottlenecks.
- **Financial KPI Cards**: Tracks Gross Revenue, Paid Orders Count, Average Order Value (AOV), and 5% Composite GST ledger.
- **Peak Kadai Frying Hours Heatmap**: Visualizes order distribution across Bengaluru meal slots ( `11-12 Lunch`, `16-17 Chai Time`, `17-19 Evening Rush`, `19-21 Dinner` ).
- **Recipe BOM & Raw Materials Depletion Table**: Displays live ingredient inventory levels, threshold alerts, unit costs, and quick restock buttons.

### Station 6: Unified Staff Login Gateway ( `/staff/login` )

- **Strict Role-Based Access Control**:
  - Unauthenticated access attempts to `/admin`, `/kds`, or `/pos` are redirected by `middleware.ts`.
  - Station selector tabs for Kitchen, Cashier, and Owner with default demo PINs ( `1122`, `3344`, `9999` ).
  - One-click instant demo authorization buttons for frictionless evaluation.

## 5. Engineering Nuances & Micro-Features ("The Small Small Things")

### A. Bengaluru PIN Code Geofencing Engine ( `lib/bengaluru-zones.ts` )

The delivery engine validates Bengaluru postal codes before checkout:

- **Inner Zone (₹25 delivery fee, ~25m delivery):** `560034` (Koramangala 1st-8th Block), `560095` (Koramangala 4th/8th Block), `560047` (Ejipura / Viveknagar).
- **Extended Zone (₹45 delivery fee, ~40m delivery):** `560038` (Indiranagar), `560102` (HSR Layout), `560068` / `560076` (BTM Layout), `560103` (Bellandur Tech Corridor), `560078` (JP Nagar),

`560029` (Tavarekere / SG Palya).

- **Out of Delivery Zone:** Gracefully rejected with clear messaging that only takeaway/counter pickup is available for addresses outside the geofenced radius.

### B. Dynamic Coupon & Promotion Engine (`lib/coupons.ts`)

Three production-grade promotion schemes with strict validation:

1. `PAV50`: Flat ₹50 off on minimum cart value of ₹199.
2. `BUMP100`: Flat ₹100 off on large group orders of ₹399+.
3. `FREECHAI`: Discount equal to 1 Kadak Cutting Chai (₹25 off) when at least two Vada Pav items are in the cart.

### C. Cryptographic Razorpay Signature Verification (`app/api/payment/verify/route.ts`)

To prevent spoofed payment confirmations, the server verifies the HMAC-SHA256 signature:

```
typescript
const generatedSignature = crypto
  .createHmac("sha256", process.env.RAZORPAY_KEY_SECRET ||
"mpc_secret_test_key_koramangala")
  .update(`${orderId}|${paymentId}`)
  .digest("hex");

const isAuthentic = generatedSignature === signature;
```

If the cryptographic signature does not match, the order remains `PENDING_PAYMENT`, an audit log entry is written, and fulfillment is blocked.

### D. Pure Web Audio Synthesizer Engine (`lib/sound.ts`)

External MP3 files often fail to play due to network latency, 404s, or mobile autoplay blocks. The app uses direct browser synthesis:

- **Order Ready Chime:** Dual harmonic chime running at 880 Hz ($A_5$ A5) and 1760 Hz ($A_6$ A6) with an exponential gain decay of 1.2 seconds.
- **Click Tick Feedback:** 1200 Hz micro-pulse running for 30 milliseconds to provide tactile touch confirmation.
- **Overdue Alarm:** 440 Hz square wave warning sound for orders exceeding prep time limits.

### E. Floating Global Live Kitchen Pulse Dock ( `components/GlobalLiveDock.tsx` )

- Sits unobtrusively at `bottom-4 right-4` on desktop and auto-collapses on screens narrower than 640px.
- Displays live active kadai frying counts, the latest ready token number, audio buzzer toggles, and direct links to active customer tickets.
- Repositioned to `bottom-20` on mobile viewports so it never overlaps the sticky mobile checkout bar.

## 6. REST API Endpoints Specification

| Method | Endpoint | Request Payload Summary | Response Summary | Auth / Role |
|---|---|---|---|---|
| `POST` | `/api/orders` | `{ customerName, customerPhone, orderType, items, tableNo, deliveryAddress, deliveryPincode, paymentMethod }` | Returns created `Order` object with unique `orderNumber` and `tokenNo`. | Public |
| `GET` | `/api/orders` | Query params: `?status=...&limit=...` | Returns list of orders sorted descending by timestamp. | Public / Staff |
| `GET` | `/api/orders/[id]` | Route parameter: `id` (Order UUID or Order Number) | Returns order details including line items, modifiers, and payment status. | Public |
| `PATCH` | `/api/orders/[id]` | `{ status: "PREPARING" \| "READY_FOR_PICKUP" \| "COMPLETED" }` | Updates ticket status, writes audit log, and adjusts inventory. | Staff |
| `POST` | `/api/payment/verify` | `{ orderId, razorpayOrderId, razorpayPaymentId, razorpaySignature }` | Verifies HMAC-SHA256 signature, marks order `PAID`, and deducts BOM stock. | Public |
| `GET` | `/api/realtime` | None (Polling bus) | Returns active kitchen orders, ready token lists, and store setting state. | Public |
| `GET` | `/api/store` | None | Returns store status, surge throttle state, and current queue depth. | Public |
| `POST` | `/api/store` | `{ isStoreOpen?: boolean, surgeThrottleActive?: boolean, bannerNotice?: string }` | Updates operational settings. | Admin Only |
| `GET` | `/api/inventory` | None | Returns list of all raw ingredients with current stock levels and threshold alerts. | Admin Only |
| `POST` | `/api/inventory` | `{ ingredientId: string, addedQty: number }` | Increments raw material stock level and logs restock event. | Admin Only |
| `POST` | `/api/auth/login` | `{ role: "kitchen" \| "cashier" \| "admin", pin?: string, identifier?: string }` | Validates credentials, sets `mpc_role` session cookie, and returns user object. | Public |

| Method | Endpoint | Request Payload Summary | Response Summary | Auth / Role |
| --- | --- | --- | --- | --- |
| `POST` | `/api/auth/logout` | None | Clears session cookies. | Public |

## 7. Complete Project Directory Structure

```
C:\Users\shiva\.gemini\antigravity\scratch\mumbai-pav-co\
├── app/
│   ├── admin/
│   │   └── page.tsx              # Store Owner HQ & Analytics Dashboard
│   ├── api/
│   │   ├── auth/
│   │   │   ├── login/route.ts    # Role authentication & session cookie handler
│   │   │   ├── logout/route.ts   # Session termination endpoint
│   │   │   ├── me/route.ts       # Active user profile inspection
│   │   │   └── register/route.ts # Customer registration with 50 loyalty points
│   │   ├── inventory/route.ts    # Recipe BOM stock levels & restock updates
│   │   ├── orders/
│   │   │   ├── [id]/route.ts     # Single order retrieval & status transitions
│   │   │   └── route.ts          # Order creation, Zod validation & listing
│   │   ├── payment/
│   │   │   └── verify/route.ts   # Razorpay cryptographic HMAC verification
│   │   ├── realtime/route.ts     # Polling bus for active fryers & tokens
│   │   └── store/route.ts        # Master store toggles & surge throttle
│   ├── kds/
│   │   └── page.tsx              # Kitchen Display System (Kanban + Timers)
│   ├── order/
│   │   └── [id]/page.tsx         # Live Digital Token Tracker screen
│   ├── pos/
│   │   └── page.tsx              # Express Counter POS Register
│   ├── staff/
│   │   └── login/page.tsx        # Unified Staff & Operations Gateway
│   ├── favicon.ico
│   ├── globals.css               # Tailwind directives & custom animations
│   ├── layout.tsx                # Viewport meta, dark root theme, font setup
│   └── page.tsx                  # Public Customer Storefront & Catalog
├── components/
│   ├── CartDrawer.tsx            # Slide-out cart, mode switcher & checkout
│   ├── CustomerAuthModal.tsx     # Customer sign-in & loyalty viewer
│   ├── GlobalLiveDock.tsx        # Floating realtime kitchen pulse widget
│   ├── MenuCustomizerModal.tsx   # Mobile bottom sheet spice & add-on customizer
│   ├── Navbar.tsx                # Customer navigation & mobile thumb bar
│   ├── StaffStationHeader.tsx    # Station role badge & station logout header
│   ├── TableQRModal.tsx          # Dine-In table selector & QR simulator
│   └── TrackOrderModal.tsx       # Quick ticket search dialog
├── lib/
│   ├── auth.ts                   # Demo credentials & client session helpers
│   ├── bengaluru-zones.ts        # Bengaluru PIN code geofence validator
│   ├── coupons.ts                # Discount coupon calculation engine
```

```
│   ├── db.ts                    # Prisma singleton instance
│   ├── sound.ts                 # Pure Web Audio synthesizer engine
│   └── types.ts                 # TypeScript interfaces & domain enums
├── prisma/
│   ├── schema.prisma            # Relational data schema (12 models)
│   └── dev.db                   # Active SQLite database file
├── public/                      # Static public assets
├── scripts/
│   └── seed.mjs                 # Database seeder with sample catalog & users
├── middleware.ts                # Next.js route guard protecting staff routes
├── package.json                 # Dependencies & npm scripts
├── postcss.config.mjs           # PostCSS plugin pipeline
├── tailwind.config.ts           # Tailwind styling theme configurations
└── tsconfig.json                # Strict TypeScript configuration
```

## 8. Operations Manual & Verification Guide

### Quick Start Commands

```
powershell
# 1. Navigate to the project root
Set-Location "C:\Users\shiva\.gemini\antigravity\scratch\mumbai-pav-co"

# 2. Synchronize Prisma database schema
npx prisma db push

# 3. Seed demo catalog, inventory & credentials
node scripts/seed.mjs

# 4. Run production compilation build
npm run build

# 5. Start development server
npm run dev
```

### Seeded Test Credentials

| Role | Email / ID | Default PIN | Target Route | Permissions |
|---|---|---|---|---|
| **Kitchen Chef** | `kitchen@mumbaipavco.in` | `1122` | `/kds` | Start frying, mark ready, print KOT, audio alerts. |
| **Counter Cashier** | `cashier@mumbaipavco.in` | `3344` | `/pos` | Quick item punch, counter cash/UPI billing, print receipts. |
| **Store Owner** | `admin@mumbaipavco.in` | `9999` | `/admin` | Store open/close, surge throttle, BOM inventory, sales ledger. |
| **Customer** | `9876543212` | *None* | `/` | Menu browsing, cart customizer, live order tracking screen. |

## 9. Security, Quality Assurance & Production Hardening

1. **Input Sanitization**: All incoming order objects are parsed against Zod schemas. Any invalid phone number, negative item quantity, or unsupported modifier is rejected with HTTP 400 before touching the database.
2. **Cryptographic Integrity**: Razorpay webhook and payment callbacks enforce HMAC-SHA256 signature matching to prevent spoofed order completion.
3. **Route Isolation**: The Next.js Edge middleware evaluates the `mpc_role` cookie on every incoming request to `/admin/*`, `/kds/*`, and `/pos/*`, preventing unauthorized customer access.
4. **Mobile Responsiveness**: Verified across 360px (narrow Android), 390px (iPhone 14/15/16), 768px (iPad portrait), and 1440px+ (desktop terminals) with responsive layouts and touch targets exceeding 44px.